

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 26 de Abril de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

1. Importe de Librerías

```
In [4]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [5]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-04-28 22:10:37.580421

2. Definición de Rutas y Parámetros

```

In [7]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
        metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
        image_height = 150
        image_width = 150
        batch_size = 15
        num_classes = None # Se determinará automáticamente del archivo CSV

```

3. Cargar y procesar los metadatos

```

In [9]: try:
        metadata = pd.read_csv(metadata_file, sep=";")
        metadata["Planta_Condición_Enfermedad"] = metadata["Planta_Estado"] + "_" + metadata["Planta_Condición_Enfermedad"]
        metadata = metadata[metadata["Planta"] != "Orange"]
        metadata = metadata[metadata["Planta"] != "Grape"]
        metadata = metadata[metadata["Planta"] != "Pepper_bell"]
        metadata = metadata[metadata["Planta"] != "Peach"]
        metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

```

```

# Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
# y 'label' (o el nombre de tu columna de etiquetas) existan.

if 'image_id' not in metadata.columns or 'Planta_Condición_Enfermedad' not in metadata.columns:
    raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Condición_Enfermedad'")
class_names = metadata['Planta_Condición_Enfermedad'].unique().tolist()
num_classes = len(class_names)
print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 37 clases: ['Yuca_sick_Cassava Bacterial Blight (CBB)', 'Yuca_sick_Cassava Mosaic Disease (CMD)', 'Yuca_sick_Cassava Brown Streak Disease (CBSD)', 'Yuca_sick_Cassava Green Mottle (CGM)', 'Yuca_healthy_healthy', 'Strawberry_healthy_healthy', 'Potato_sick_Early_blight', 'Corn_(maize)_healthy_healthy', 'Tomato_sick_Target_Spot', 'Potato_sick_Late_blight', 'Tomato_sick_Late_blight', 'Tomato_sick_Tomato_mosaic_virus', 'Tomato_sick_Leaf_Mold', 'Apple_sick_Cedar_apple_rust', 'Tomato_sick_Bacterial_spot', 'Tomato_sick_Early_blight', 'Corn_(maize)_sick_Common_rust', 'Tomato_healthy_healthy', 'Tomato_sick_Tomato_Yellow_Leaf_Curl_Virus', 'Apple_sick_Apple_scab', 'Corn_(maize)_sick_Northern_Leaf_Blight', 'Tomato_sick_Spider_mites Two-spotted_spider_mite', 'Tomato_sick_Septoria_leaf_spot', 'Corn_(maize)_sick_Cercospora_leaf_spot Gray_leaf_spot', 'Apple_sick_Black_rot', 'Apple_healthy_healthy', 'Strawberry_sick_Leaf_scorch', 'Potato_healthy_healthy', 'cauliflower_sick_downy_mildew', 'cucumber_healthy_healthy', 'bean_sick_angular_leaf_spot', 'cauliflower_healthy_healthy', 'cauliflower_sick_black_rot', 'bean_sick_rust', 'cauliflower_sick_bacterial_spot_rot', 'bean_healthy_healthy', 'cucumber_sick_diseased']

```
In [10]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
train.shape, test.shape
```

```
Out[10]: ((42354, 8), (10589, 8))
```

4. Crear un generador de datos

```
In [12]: # Función de preprocesamiento usando solo TensorFlow estándar
def preprocess_image(img):
    img = tf.image.random_brightness(img, max_delta=0.2) # Variación de brillo
    img = tf.image.random_contrast(img, lower=0.8, upper=1.2) # Variación de contraste

    # Añadir un poco de ruido gaussiano
    noise = tf.random.normal(shape=tf.shape(img), mean=0.0, stddev=0.02, dtype=tf.float32)
    img = img + noise

```

```
img = tf.clip_by_value(img, 0.0, 1.0) # Mantener los valores entre 0 y 1
return img

# Generador de datos
datagen = ImageDataGenerator(
    validation_split=0.2,
    preprocessing_function=preprocess_image,
    rotation_range=45,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    fill_mode='nearest',
    shear_range=0.1,
    height_shift_range=0.1,
    width_shift_range=0.1,
    brightness_range=[0.8, 1.2]
)

# Generador para entrenamiento
train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Condición_Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='training',
    seed=42,
    classes=class_names,
    shuffle=True
)

# Generador para validación
validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Condición_Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names,
    shuffle=False
)
```

Found 33880 validated image filenames belonging to 37 classes.
Found 8470 validated image filenames belonging to 37 classes.

5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [14]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

6. Congelar las capas convolucionales base

```
In [16]: for layer in base_model.layers:  
        layer.trainable = False
```

7. Añadir una nueva capa de clasificación personalizada

```
In [18]: x = base_model.output  
x = GlobalAveragePooling2D()(x)  
# Puedes añadir más capas densas aquí si lo deseas  
# x = Dense(1024, activation='relu')(x)  
predictions = Dense(num_classes, activation='softmax')(x)
```

8. Crear el modelo final

```
In [20]: model = Model(inputs=base_model.input, outputs=predictions)
```

9. Compilar el modelo para fine-tuning de la última capa

```
In [22]: model.compile(optimizer='adam',  
                      loss='categorical_crossentropy',  
                      metrics=['accuracy'])
```

10. Entrenar el modelo con la última capa descongelada

```
In [24]: epochs_fine_tune = 20 # Define el número de épocas para el fine-tuning
```

```
In [25]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    verbose = 1
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/20

2258/2258  **772s** 341ms/step - accuracy: 0.2459 - loss: 3.0188 - val_accuracy: 0.2591 - val_loss: 2.8991

Epoch 2/20

2258/2258  **162s** 72ms/step - accuracy: 0.4000 - loss: 2.4789 - val_accuracy: 0.2593 - val_loss: 2.9032

Epoch 3/20

2258/2258  **834s** 369ms/step - accuracy: 0.2598 - loss: 2.9229 - val_accuracy: 0.2624 - val_loss: 2.8948

Epoch 4/20

2258/2258  **167s** 74ms/step - accuracy: 0.3333 - loss: 2.5065 - val_accuracy: 0.2623 - val_loss: 2.9010

Epoch 5/20

2258/2258  **840s** 372ms/step - accuracy: 0.2640 - loss: 2.8977 - val_accuracy: 0.2845 - val_loss: 2.8230

Epoch 6/20

2258/2258  **165s** 73ms/step - accuracy: 0.3333 - loss: 2.4951 - val_accuracy: 0.2797 - val_loss: 2.8319

Epoch 7/20

2258/2258  **836s** 370ms/step - accuracy: 0.2756 - loss: 2.8622 - val_accuracy: 0.2908 - val_loss: 2.8468

Epoch 8/20

2258/2258  **167s** 74ms/step - accuracy: 0.2667 - loss: 2.8890 - val_accuracy: 0.2869 - val_loss: 2.8524

Epoch 9/20

2258/2258  **846s** 375ms/step - accuracy: 0.2818 - loss: 2.8480 - val_accuracy: 0.2862 - val_loss: 2.8036

Epoch 10/20

2258/2258  **174s** 77ms/step - accuracy: 0.3333 - loss: 2.7067 - val_accuracy: 0.2798 - val_loss: 2.8172

Epoch 11/20

2258/2258  **877s** 389ms/step - accuracy: 0.2812 - loss: 2.8476 - val_accuracy: 0.2922 - val_loss: 2.8333

Epoch 12/20

2258/2258  **175s** 77ms/step - accuracy: 0.3333 - loss: 2.7163 - val_accuracy: 0.2953 - val_loss: 2.8400

Epoch 13/20

2258/2258  **872s** 386ms/step - accuracy: 0.2798 - loss: 2.8454 - val_accuracy: 0.2953 - val_loss: 2.8038

Epoch 14/20
2258/2258 ————— **172s** 76ms/step - accuracy: 0.5333 - loss: 2.1596 - val_accuracy: 0.2969 - val_loss: 2.8054
 Epoch 15/20
2258/2258 ————— **865s** 383ms/step - accuracy: 0.2886 - loss: 2.8223 - val_accuracy: 0.2878 - val_loss: 2.8048
 Epoch 16/20
2258/2258 ————— **170s** 75ms/step - accuracy: 0.2000 - loss: 3.1616 - val_accuracy: 0.2890 - val_loss: 2.7939
 Epoch 17/20
2258/2258 ————— **848s** 375ms/step - accuracy: 0.2892 - loss: 2.8195 - val_accuracy: 0.2859 - val_loss: 2.7973
 Epoch 18/20
2258/2258 ————— **165s** 73ms/step - accuracy: 0.3333 - loss: 2.7565 - val_accuracy: 0.2857 - val_loss: 2.7978
 Epoch 19/20
2258/2258 ————— **835s** 370ms/step - accuracy: 0.2860 - loss: 2.8183 - val_accuracy: 0.2864 - val_loss: 2.7890
 Epoch 20/20
2258/2258 ————— **166s** 73ms/step - accuracy: 0.1333 - loss: 3.0304 - val_accuracy: 0.2869 - val_loss: 2.7846

```
In [26]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-04-29 00:59:06.191988

Duración: 2:48:28.611567

11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [28]: # unfreeze_layers = 5 # Número de capas a descongelar (contando desde la última)

# for layer in base_model.layers[-unfreeze_layers:]:
#     layer.trainable = True
```

12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [30]: #model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Usar Adam
#                   loss='categorical_crossentropy',
#                   metrics=['accuracy'])
```

13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [32]: #epochs_deep_fine_tune = 5 # Define el número de épocas para el fine-tuning

# print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layer}
# history_deep_fine_tune = model.fit(
#     train_generator,
#     steps_per_epoch=train_generator.samples // train_generator.batch_size,
#     validation_data=validation_generator,
#     validation_steps=validation_generator.samples // validation_generator.b
#     epochs=epochs_deep_fine_tune
# )
```

14. Evaluar el modelo final

```
In [34]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

565/565  **166s** 293ms/step - accuracy: 0.2886 - loss: 2.7766

Pérdida en la validación: 2.7897

Precisión en la validación: 0.2857

```
In [35]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

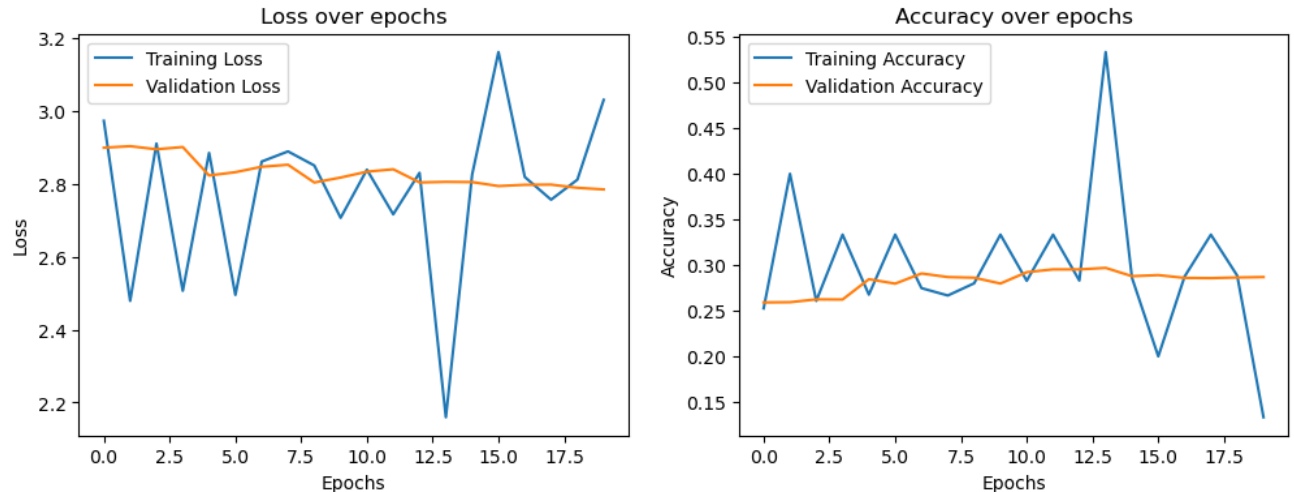
# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
```



```
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[35]: <matplotlib.legend.Legend at 0x295364140>



```
In [36]: epochs = list(range(1, 21))
data = {
    'Epoch': epochs,
    'Training Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	2.972895	2.899111	0.252591
1	2	2.478927	2.903223	0.400000
2	3	2.910513	2.894783	0.260741
3	4	2.506467	2.900963	0.333333
4	5	2.884914	2.822966	0.267651
5	6	2.495142	2.831901	0.333333
6	7	2.861087	2.846806	0.274738
7	8	2.888958	2.852386	0.266667
8	9	2.850263	2.803581	0.280024
9	10	2.706690	2.817193	0.333333
10	11	2.838847	2.833343	0.282770
11	12	2.716342	2.839962	0.333333
12	13	2.830196	2.803792	0.283036
13	14	2.159602	2.805390	0.533333
14	15	2.825719	2.804808	0.286107
15	16	3.161586	2.793904	0.200000
16	17	2.818755	2.797294	0.287435
17	18	2.756477	2.797829	0.333333
18	19	2.811487	2.789030	0.288439
19	20	3.030384	2.784633	0.133333

	Validation Accuracy
0	0.259102
1	0.259338
2	0.262411
3	0.262293
4	0.284515
5	0.279669
6	0.290780
7	0.286879
8	0.286170
9	0.279787
10	0.292199
11	0.295272
12	0.295272
13	0.296927
14	0.287825
15	0.289007
16	0.285934
17	0.285697
18	0.286407
19	0.286879

```
In [37]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [38]: print('Perdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])
```

```
print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accuracy'][-1])
```

Perdida final en Train: 3.030383825302124
 Pérdida final en Validation: 2.784632682800293
 Accuracy final en Train: 0.13333334028720856
 Accuracy final en Validation: 0.28687942028045654

In []:

15. Testeo del Modelo

```
In [40]: test.Planta = test.Planta.astype('str')

test_generator = datagen.flow_from_dataframe(test,
                                             directory = "Datos Finales",
                                             subset = "training",
                                             x_col = "image_id",
                                             y_col = "Planta Condición Enfermedad",
                                             target_size = (image_height, image_width),
                                             batch_size = batch_size,
                                             class_mode = "categorical")
```

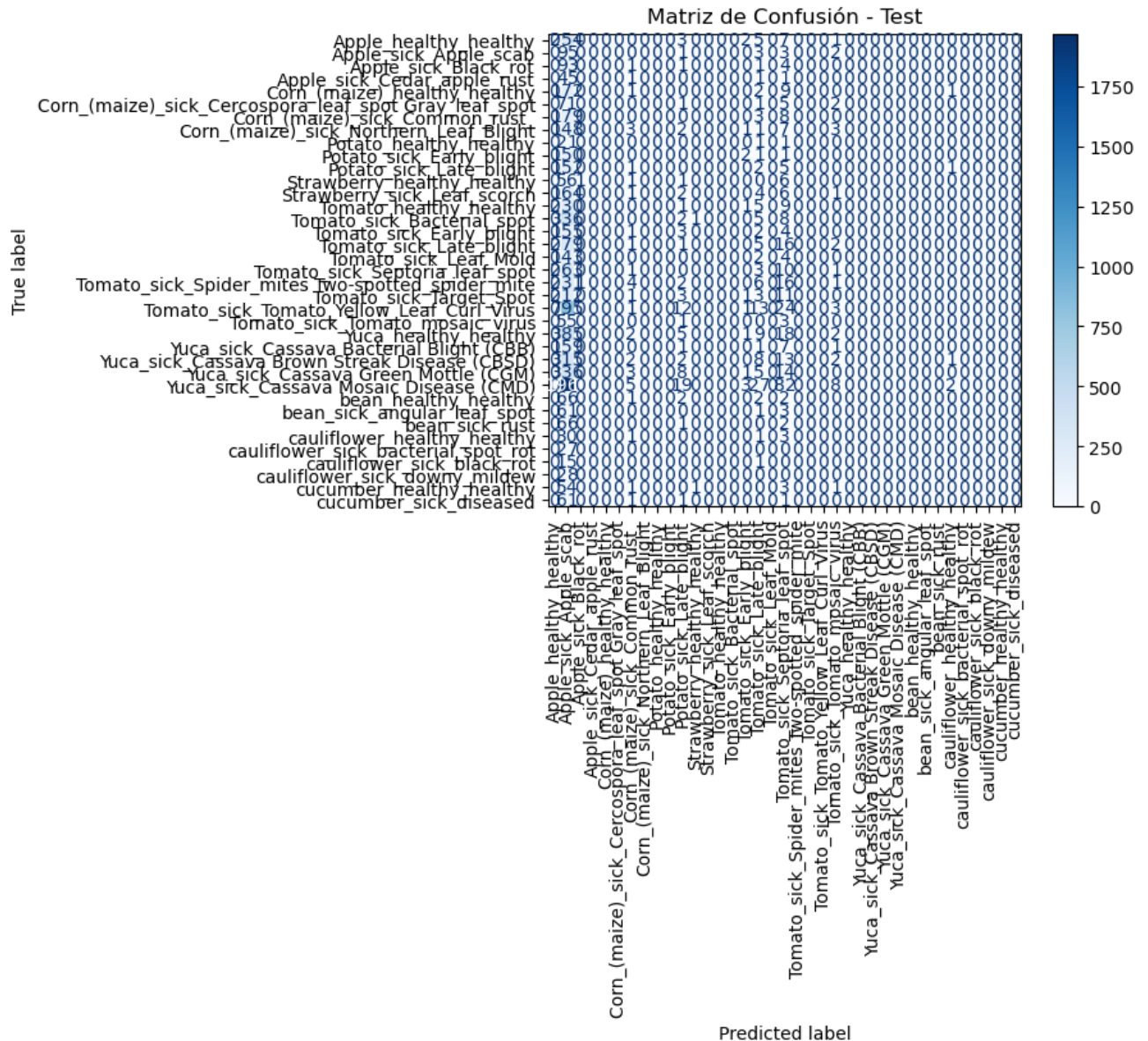
Found 8472 validated image filenames belonging to 37 classes.

```
In [41]: # -----
# 1. Obtener predicciones completas para test
# -----
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

565/565  167s 294ms/step

```
In [42]: # -----
# 2. Matriz de Confusión
# -----
plt.figure(figsize=(200, 80))
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_generator.classes)
disp.plot(cmap="Blues", xticks_rotation=90)
plt.title("Matriz de Confusión - Test")
plt.savefig("matriz_confusion_test_modelo_11.png", dpi=300, bbox_inches='tight')
plt.show()
```

<Figure size 20000x8000 with 0 Axes>



```
In [43]: accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.0130

```
In [44]: # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())
```

```
# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.05)
plt.show()
```

1/1 ————— 1s 799ms/step

```

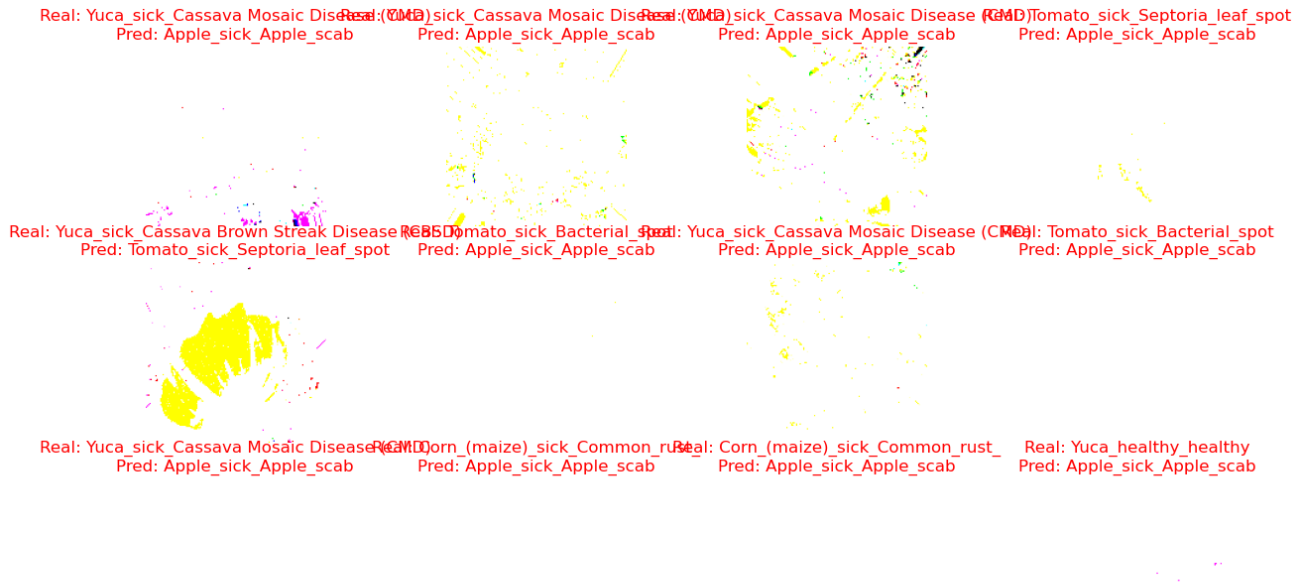
-----
ValueError                                Traceback (most recent call last)
Cell In[44], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
----> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```



```
In [ ]: # 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
In [ ]: from sklearn.metrics import roc_auc_score
```

```
# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
In [ ]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

16. Guardar el modelo entrenado

```
In [ ]: model.save('modelo_12_resnet50_fine_tuned.h5')
```

```
In [ ]:
```

```
In [ ]:
```